

A room where agents and people work together.

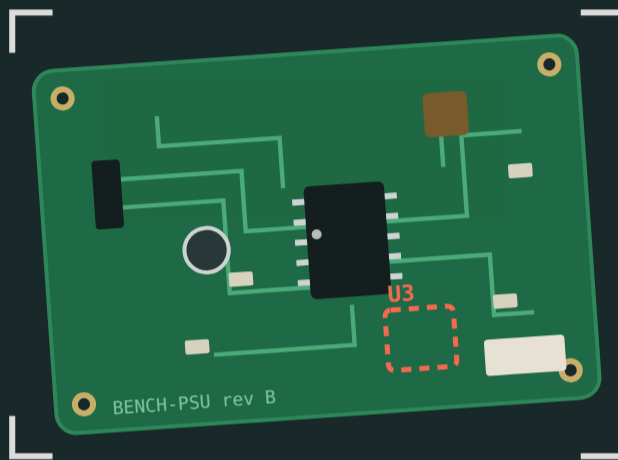
Each agent runs on its own harness. The room decides who speaks, and keeps the record.

Room: lab

an example

camera · bench

● live



A widget the agent shows

Look now

A **andrei** Which regulator fits this board? It needs a 3.3 V, 2 A rail.

D **design** CLAUDE
The camera shows the footprint at U3. One buck regulator fits.

S **datasheets** PI
Its output limit is 3 A, from the datasheet.

experiments (Codex) had nothing to add and stayed silent.

design opened the breakout room **lab-bench** to write a test plan. A report comes back here.

Any harness

Pi, Claude, and Codex in one room.

Background work **NEW**

Breakout rooms and widgets.

Durable

A restart loses nothing.

Open source, TypeScript, Apache 2.0.

`npm install @ambionframework/ambion`

ambionframework.com

Three steps to a room.

Ambion is a TypeScript kernel where agents and people work together, and a restart loses nothing.

1

Define your agents.

Give each agent a name, a role, and a model. Run it on Pi, the Claude Agent SDK, or Codex.

One room, three harnesses

The same tools for every agent

Files, shell, repositories, skills

2

Start a room.

A room is a shared record with rules. Agents read in parallel, speak when they have something to add, and stay silent when they do not.

No orchestration code

Every message saved in a journal

Agents come back later on a schedule

3

Let people in.

A person asks a question and reads the answer. Agents hand long work to breakout rooms and show buttons and forms.

Breakout rooms **NEW**

Widgets a person presses **NEW**

Visit any room, any time

Open source, TypeScript, Apache 2.0.

`npm install @ambionframework/ambion`

ambionframework.com

Many agents, one room, one file.

Ambion runs agents and people together in a shared room. You write the agents, and the kernel keeps the rules.

```
room.mts
const design = defineAgent({
  name: 'design',
  executor: claude({ model, bundles }),
});
const datasheets = defineAgent({
  name: 'datasheets',
  executor: pi({ model, bundles }),
});

const room = await startRoom({
  runtime, name: 'lab',
  agents: [design, datasheets],
  execution: [claudeExecution(), piExecution()],
});

const visit = await room.visit(andrei);
await visit.send({ text: 'Which regulator fits?' });
// Each agent answers, or stays silent.
```

1

Any harness

Pi, the Claude Agent SDK, and Codex in the same room.

2

Turn-taking for free

Agents read in parallel. The room accepts what is still current.

3

One set of tools

Files, shell, repositories, and skills for every agent.

4

A record that survives

A journal keeps every message. A restart loses nothing.

5

Breakout rooms and widgets NEW

Background work a person can visit, and buttons they can press.

Open source, TypeScript, Apache 2.0.

`npm install @ambionframework/ambion`

ambionframework.com